

LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 6

Student Name: Mohammed Almas

Roll No.: A24126552098

Date: 05-10-2026

1. TITLE

Mini Application using Express and Node.js

2. AIM

To develop a simple Bus Reservation System mini application using Node.js and Express.js that allows users to view available buses and book a bus ticket through a web interface.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the basics of Node.js and Express.js.
 - To create a web server using Express.
 - To serve HTML, CSS and JavaScript files using `express.static()`.
 - To implement GET and POST routes.
 - To retrieve and display available bus details.
 - To accept passenger booking information from a web form.
 - To exchange data between the browser and server using `fetch()`.
 - To display a successful ticket booking confirmation.
-

4. THEORY

Node.js: Node.js is a JavaScript runtime environment that allows JavaScript to run outside a web browser and is commonly used for server-side applications.

Express.js: Express.js is a web application framework for Node.js that simplifies server creation, routing and handling HTTP requests and responses.

Express Application: The `express()` function creates an Express application instance. Routes such as GET and POST can then be defined.

Middleware: Middleware functions process requests before the final route handler. This application uses `express.json()` and `express.urlencoded()` to process request data.

Static Files: `express.static('public')` makes HTML, CSS and JavaScript files inside the public folder available to the browser.

GET Route: The `/buses` GET route returns the list of available buses as JSON data.

POST Route: The `/book` POST route receives passenger name, bus number and seat number and returns a booking confirmation.

fetch(): JavaScript `fetch()` is used by the browser to communicate with the Express server and process JSON responses.

JSON: JSON provides a simple format for exchanging structured data between the client and server.

Port 3000: The Express application listens on port 3000 and can be accessed using `http://localhost:3000`.

5. ALGORITHM / PROCEDURE

1. Install Node.js and verify the Node.js and npm versions.
 2. Create a project folder named busReservation.
 3. Initialize the Node.js project and install Express using npm.
 4. Create server.js and import the Express module.
 5. Create an Express application and configure JSON and URL-encoded middleware.
 6. Serve the files in the public folder using `express.static()`.
 7. Define the root GET route to display index.html.
 8. Define the `/buses` GET route to return the available bus details as JSON.
 9. Define the `/book` POST route to receive passenger booking data.
 10. Create index.html containing the bus display and ticket booking form.
 11. Create styles.css for webpage presentation and script.js for client-side operations.
 12. Use `fetch('/buses')` to retrieve available buses from the server.
 13. Use `fetch('/book')` with POST to submit booking details.
 14. Start the Express server on port 3000 using `node server.js`.
 15. Open `http://localhost:3000` in a web browser.
 16. Verify the displayed buses and successful ticket booking output.
-

6. PROGRAM

server.js

```
const express=require("express");
const app=express();

app.use(express.json());
app.use(express.urlencoded({extended:true}));
app.use(express.static("public"));

app.get("/",(req,res)=>{
    res.sendFile(__dirname+"/public/index.html");
});

app.get("/buses",(req,res)=>{
    const buses=[
        {busNo:"BUS101",source:"Hyderabad",destination:"Bangalore",seats:20},
        {busNo:"BUS102",source:"Hyderabad",destination:"Chennai",seats:15},
        {busNo:"BUS103",source:"Hyderabad",destination:"Mumbai",seats:25}
    ];
    res.json(buses);
});

app.post("/book",(req,res)=>{
    const {name,busNo,seatNo}=req.body;
    res.json({
        success:true,
        message:"Ticket booked successfully!",
        passenger:name,bus:busNo,seat:seatNo
    });
});

app.listen(3000,()=>{
    console.log("Bus Reservation Server started");
    console.log("Open http://localhost:3000");
});
```

6. PROGRAM (CONTINUED)

public/index.html

```
<!DOCTYPE html>
<html>
<head>
<title>Bus Reservation System</title>
<link rel="stylesheet" href="styles.css">
</head>
<body>
<div class="container">
<h1>Bus Reservation System</h1>
<h2>Available Buses</h2>
<button onclick="loadBuses()">View Buses</button>
<div id="busList"></div>
<h2>Book Your Ticket</h2>
<form id="bookingForm">
<label>Passenger Name:</label>
<input type="text" id="name" required>
<label>Bus Number:</label>
<input type="text" id="busNo" required>
<label>Seat Number:</label>
<input type="number" id="seatNo" required>
<button type="submit">Book Ticket</button>
</form>
<div id="result"></div>
</div>
<script src="script.js"></script>
</body>
</html>
```

public/styles.css

```
body{font-family:Arial;background:#f2f2f2;padding:30px}
.container{width:600px;margin:auto;background:white;padding:25px;border-radius:10px}
h1{text-align:center}
button{padding:10px 15px;background:#333;color:white;border:none;border-radius:5px;cursor:pointer}
input{display:block;width:95%;padding:10px;margin:8px 0 15px}
.bus{background:#eeeeee;padding:12px;margin:10px 0;border-radius:5px}
#result{margin-top:20px;padding:15px;background:#e8f5e9}
```

public/script.js

```
function loadBuses(){
  fetch("/buses").then(response=>response.json()).then(buses=>{
    let output="";
    buses.forEach(bus=>{
      output+=`<div class="bus"><b>Bus Number:</b> ${bus.busNo}<br>
<b>From:</b> ${bus.source}<br><b>To:</b> ${bus.destination}<br>
<b>Available Seats:</b> ${bus.seats}</div>`;
    });
    document.getElementById("busList").innerHTML=output;
  });
}

document.getElementById("bookingForm").addEventListener("submit",function(event){
  event.preventDefault();
  const name=document.getElementById("name").value;
  const busNo=document.getElementById("busNo").value;
  const seatNo=document.getElementById("seatNo").value;
  fetch("/book",{method:"POST",headers:{"Content-Type":"application/json"},
  body:JSON.stringify({name:name,busNo:busNo,seatNo:seatNo})})
  .then(response=>response.json()).then(data=>{
    document.getElementById("result").innerHTML=
    `<h3>${data.message}</h3><p><b>Passenger:</b> ${data.passenger}</p>
    <p><b>Bus:</b> ${data.bus}</p><p><b>Seat:</b> ${data.seat}</p>`;
  });
});
```

7. CODE EXPLANATION

Code	Explanation
<code>const express=require("express");</code>	Imports the Express framework.
<code>express()</code>	Creates an Express application instance.
<code>express.json()</code>	Parses incoming JSON request data.
<code>express.static("public")</code>	Serves HTML, CSS and JavaScript files from the public folder.
<code>app.get("/", ...)</code>	Defines the root GET route and sends index.html.
<code>app.get("/buses", ...)</code>	Returns the available bus details as JSON.
<code>res.json(buses)</code>	Sends the bus array as a JSON response.
<code>app.post("/book", ...)</code>	Defines the POST route used for ticket booking.
<code>req.body</code>	Contains the passenger name, bus number and seat number sent by the client.
<code>res.json({...})</code>	Sends the booking confirmation back to the browser.
<code>app.listen(3000, ...)</code>	Starts the Express server on port 3000.

7. CODE EXPLANATION (CONTINUED)

Code	Explanation
<code>fetch('/buses')</code>	Requests available bus data from the Express GET route.
<code>response.json()</code>	Converts the server response into JavaScript JSON data.
<code>buses.forEach()</code>	Loops through each bus and prepares its details for display.
<code>innerHTML</code>	Places the generated bus details into the webpage.
<code>fetch("/book",{method:"POST"})</code>	Sends booking data to the Express POST route.
<code>Content-Type: application/json</code>	Indicates that the request body contains JSON data.
<code>JSON.stringify()</code>	Converts the booking object into a JSON string.
<code>event.preventDefault()</code>	Prevents the browser from refreshing the page after form submission.
<code>styles.css</code>	Provides the visual styling for the Bus Reservation System page.
<code>index.html</code>	Provides the webpage structure and ticket booking form.
<code>script.js</code>	Handles bus retrieval and ticket booking using <code>fetch()</code> .

8. TEST CASES AND EXECUTION DATA

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Run node server.js	Server should start on port 3000.	Server started and localhost:3000 displayed.	Pass
TC-02	Open /	Bus Reservation System page should be displayed.	Home page displayed successfully.	Pass
TC-03	Click View Buses	Available buses should be displayed.	BUS101, BUS102 and BUS103 displayed.	Pass
TC-04	Open /buses	Server should return bus details as JSON.	Three bus records returned.	Pass
TC-05	Enter passenger, bus and seat details	Booking request should be submitted.	Booking confirmation displayed.	Pass
TC-06	POST /book	Server should return booking confirmation JSON.	Ticket booked successfully response received.	Pass
TC-07	Refresh/open localhost:3000	Application should remain accessible.	Express application accessible.	Pass

9. OUTPUT

Output 1: Express server started successfully on port 3000.

```
PS D:\FULL STACK GITHUB\Records\week-7-8> cd busReservation
PS D:\FULL STACK GITHUB\Records\week-7-8\busReservation> node server.js
Bus Reservation Server started
Open http://localhost:3000
```

Output 2: Bus Reservation System home page displayed.

← → http://localhost:3000/

Bus Reservation System

Available Buses

[View Buses](#)

Book Your Ticket

Passenger Name: Bus Number: Seat Number: [Book Ticket](#)

Output 3: Available buses displayed.

Bus Reservation System

Available Buses

[View Buses](#)

Bus Number: BUS101
From: Hyderabad
To: Bangalore
Available Seats: 20

Bus Number: BUS102
From: Hyderabad
To: Chennai
Available Seats: 15

Bus Number: BUS103
From: Hyderabad
To: Mumbai
Available Seats: 25

Output 4: /buses JSON response.

← → http://localhost:3000/buses

Pretty-print ☒

```
[
  {
    "busNo": "BUS101",
    "source": "Hyderabad",
    "destination": "Bangalore",
    "seats": 20
  },
  {
    "busNo": "BUS102",
    "source": "Hyderabad",
    "destination": "Chennai",
    "seats": 15
  },
  {
    "busNo": "BUS103",
    "source": "Hyderabad",
    "destination": "Mumbai",
    "seats": 25
  }
]
```

Output 5: Ticket booking confirmation displayed.

Book Your Ticket

Passenger Name: Bus Number: Seat Number: [Book Ticket](#)

Ticket booked successfully!

Passenger: almas

Bus: 2020

Seat: 20

10. RESULT

Thus, a Bus Reservation System mini application was successfully developed using Node.js and Express.js. The application demonstrates Express server creation, middleware, static file serving, GET and POST routing, JSON data exchange, browser-server communication using `fetch()`, display of available buses, and successful ticket booking.
